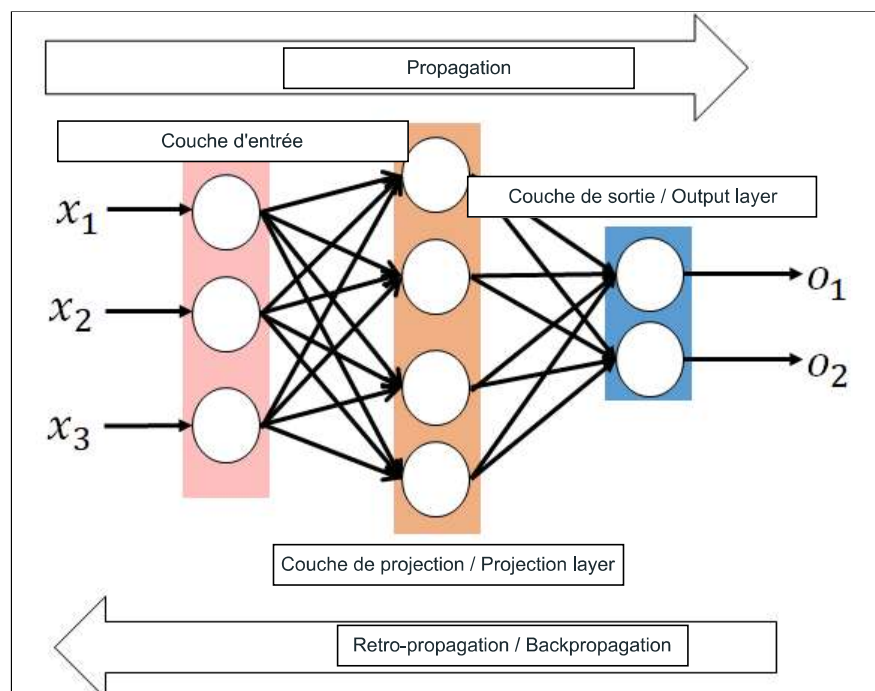


Commencé le lundi 10 mars 2025, 13:21**État** Terminé**Terminé le** lundi 10 mars 2025, 13:25**Temps mis** 3 min 43 s**Note** 9,00 sur 20,00 (45%)**Question 1**

Correct

Note de 3,00 sur 3,00

Faites glisser au bon endroit.



Apprentissage / Learning

variables entrées / input variables

Your answer is correct.

Question 2

Correct

Note de 5,00 sur 5,00

Dans un réseau de neurones à 2 couches, l'algorithme d'apprentissage suit 5 étapes successives. Pour chaque étape, entrez dans le champ correspondant le numéro de la description qui lui correspond.

Descriptions :

1. Retour en 1 ou arrêt
2. Mise à jour des poids de la deuxième couche
3. Propagation de x dans le réseau
4. Calcul de l'erreur quadratique et du gradient
5. Mise à jour des poids de la première couche

De plus, précisez si cet algorithme est "simple" ou "complex", et s'il est "resource-intensive" ou "not resource-intensive" en ressources.

Étape 1 : ✓Étape 2 : ✓Étape 3 : ✓Étape 4 : ✓Étape 5 : ✓Algorithme : ✓Ressources : ✓

L'algorithme de rétropropagation du gradient (backpropagation) est une technique incontournable qu'il faut connaître et maîtriser. Il est au cœur de nombreux projets d'apprentissage automatique, en particulier dans l'entraînement des réseaux de neurones artificiels.

Question 3

Correct

Note de 1,00 sur 1,00

Laquelle des approches suivantes peut être utilisée pour traiter un jeu de données avec des classes déséquilibrées ?

Veuillez choisir une réponse.

- ☒ a. Toutes les réponses ci-dessus. ✓ Correct, les trois méthodes mentionnées sont valides pour traiter des classes déséquilibrées.
- ☐ b. Réduction de la classe majoritaire.
- ☐ c. Augmentation de la classe minoritaire.
- ☐ d. Génération de données synthétiques.

Lorsque vous travaillez avec un jeu de données déséquilibré, plusieurs approches peuvent être utilisées pour traiter ce déséquilibre :

- **Sous-échantillonnage de la classe majoritaire** : Cette méthode consiste à réduire le nombre d'exemples de la classe majoritaire pour équilibrer le nombre d'exemples entre les différentes classes.
- **Sur-échantillonnage de la classe minoritaire** : Cette approche consiste à augmenter le nombre d'exemples de la classe minoritaire en dupliquant des exemples ou en générant des exemples synthétiques (par exemple avec la méthode SMOTE).
- **Pondération des classes** : Certaines méthodes d'apprentissage peuvent pondérer différemment les erreurs commises sur les classes minoritaires et majoritaires, de manière à donner plus de poids à la classe minoritaire.
- **Utilisation d'algorithmes adaptés aux classes déséquilibrées** : Certains algorithmes sont conçus pour mieux traiter les déséquilibres de classes (comme les forêts aléatoires ou les réseaux de neurones avec des mécanismes de pondération de classes).

La normalisation des données et la réduction de la dimensionnalité ne sont pas directement liées au traitement des classes déséquilibrées.

La réponse correcte est : Toutes les réponses ci-dessus.

Question 4

Non répondue

Noté sur 3,00

Un modèle prédit les valeurs suivantes pour un jeu de données : $\hat{y} = [2, 6, 2]$. Les valeurs cibles correspondantes sont $y = [4, 4, 1]$. Calculez l'erreur quadratique moyenne (MSE).

Réponse : ✖

L'erreur quadratique moyenne (MSE) est une mesure couramment utilisée pour évaluer la précision d'un modèle de régression. Elle mesure la moyenne des carrés des erreurs, où chaque erreur est la différence entre la valeur prédite et la valeur réelle.

La formule de l'erreur quadratique moyenne est la suivante :

$$MSE = (1/n) * \sum (\hat{y}_i - y_i)^2$$

Dans ce cas, nous avons :

- $\hat{y} = [2, 6, 2]$
- $y = [4, 4, 1]$

Le calcul de l'erreur quadratique pour chaque élément est :

- $(2 - 4)^2 = 4$
- $(6 - 4)^2 = 4$
- $(2 - 1)^2 = 1$

L'erreur quadratique moyenne est donc :

$$MSE = (4 + 4 + 1) / 3 = 9 / 3 = 3.3333$$

Il est important de noter que le MSE pénalise fortement les erreurs plus grandes en raison de l'élévation au carré. Cela fait du MSE une métrique sensible aux valeurs aberrantes.

La réponse correcte est : 3

Question 5

Non répondue

Noté sur 1,00

Quelle méthode peut aider à éviter le sur-apprentissage dans un réseau de neurones ?

Veuillez choisir une réponse.

- ☐ a. Arrêt précoce de l'apprentissage (early stopping). {mlang en}Early stoppin
- ☐ b. Réduction de la base d'apprentissage.
- ☐ c. Augmentation de la complexité du modèle.
- ☐ d. Utilisation du gradient total uniquement.

Le sur-apprentissage (ou overfitting) survient lorsque le modèle apprend trop bien les détails et le bruit de l'ensemble d'entraînement, ce qui l'empêche de généraliser correctement sur des données nouvelles. Pour éviter ce problème, plusieurs méthodes peuvent être utilisées :

- **Régularisation L2** (ou pénalité de poids) : Elle consiste à ajouter une pénalité à la fonction de coût, proportionnelle au carré de la norme des poids. Cela réduit la taille des poids et empêche le modèle de s'adapter trop précisément aux données d'entraînement, ce qui aide à généraliser mieux sur des données non vues.
- **Dropout** : Cette méthode consiste à "désactiver" aléatoirement une proportion de neurones durant l'entraînement à chaque itération, afin d'empêcher le réseau de devenir trop dépendant d'un petit sous-ensemble de neurones, ce qui améliore la robustesse du modèle et évite le sur-apprentissage.
- **Augmentation des données** : Cette technique consiste à augmenter la taille de l'ensemble d'entraînement en créant des variations artificielles des données existantes (par exemple, en faisant des rotations ou des translations d'images). Cela permet de donner au modèle une plus grande diversité d'exemples et d'éviter le sur-ajustement aux données spécifiques.
- **Réduction de la taille du modèle** : Un modèle trop complexe, avec trop de paramètres, est plus susceptible de sur-apprendre. En réduisant la taille du modèle (par exemple, en diminuant le nombre de couches ou de neurones), on limite sa capacité à mémoriser les données d'entraînement, ce qui aide à éviter le sur-apprentissage.

Il est important de noter que l'une ou l'autre de ces techniques peut être utilisée seule, mais souvent, une combinaison de plusieurs méthodes est la plus efficace pour éviter le sur-apprentissage et améliorer la généralisation du modèle.

La réponse correcte est : Arrêt précoce de l'apprentissage (early stopping). {mlang en}Early stoppin

Question 6

Non répondue

Noté sur 1,00

Un modèle avec un biais faible et une variance élevée est généralement :

Veuillez choisir une réponse.

- ☐ a. Sur-dimensionné.
- ☐ b. Sous-dimensionné.
- ☐ c. Bien dimensionné.
- ☐ d. Non optimisé.

Un modèle avec un biais faible et une variance élevée est généralement le signe d'un **surajustement** (ou overfitting). Dans ce cas, le modèle a appris les détails et le bruit de l'ensemble d'entraînement au point de ne plus être capable de généraliser correctement aux nouvelles données. Il s'ajuste très précisément aux données d'entraînement, mais sa performance sur des données de test ou sur de nouvelles données est médiocre.

Le biais faible indique que le modèle est capable de bien s'ajuster aux données d'entraînement, ce qui signifie qu'il n'ignore pas les relations importantes dans les données. Cependant, la variance élevée signifie que le modèle est trop sensible aux petites variations dans l'ensemble d'entraînement, ce qui peut entraîner une sur-adaptation aux spécificités de ces données.

En revanche, un modèle avec un biais élevé et une variance faible est souvent sous-ajusté, c'est-à-dire qu'il ne parvient pas à capturer les relations importantes dans les données d'entraînement. Un bon modèle doit trouver un équilibre entre biais et variance pour généraliser efficacement aux nouvelles données.

La réponse correcte est : Sur-dimensionné.

Question 7

Non répondue

Noté sur 3,00

On souhaite reconnaître des images de chiffres de résolution 30x30.

On utilise un réseau de neurones monocouche pour réaliser cette tâche.

Combien de paramètres libres (poids et biais) sont nécessaires ?

Réponse : ✖

Le calcul du nombre de paramètres libres dans un réseau de neurones monocouche se fait en tenant compte des poids et des biais. Pour un réseau avec 30x30 pixels en entrée et 10 classes de chiffres en sortie, on calcule :

- Poids = Nombre d'entrées (30 x 30) x Nombre de sorties (10) = 900 x 10 = 9 000
- Biais = Nombre de sorties (10) = 10
- Donc, le total des paramètres libres est 9 000 + 10 = 9 010.

Si votre réponse est correcte, félicitations ! Si ce n'est pas le cas, revérifiez votre calcul, en particulier l'intégration des poids et des biais dans le réseau.

La réponse correcte est : 9010

Question 8

Non répondue

Noté sur 1,00

Quelle méthode est utilisée pour centrer et réduire les données d'entrée dans un réseau de neurones ?

Veuillez choisir une réponse.

- ☐ a. $x' = (x - \mu) / \sigma$
- ☐ b. $x' = x - \mu$
- ☐ c. $x' = x / \sigma$
- ☐ d. $x' = (x + \mu) \times \sigma$

La normalisation des données améliore la convergence des algorithmes d'apprentissage.

La réponse correcte est : $x' = (x - \mu) / \sigma$

Question 9

Non répondue

Noté sur 1,00

Quelle méthode d'initialisation des poids est recommandée pour éviter la saturation dans un réseau de neurones ?

Veuillez choisir une réponse.

- ☐ a. Initialisation par Xavier.
- ☐ b. Initialisation aléatoire entre -1 et 1.
- ☐ c. Initialisation avec des poids nuls
- ☐ d. Initialisation à partir de l'échantillon d'apprentissage.

Lors de l'initialisation des poids dans un réseau de neurones, il est crucial de choisir une méthode qui évite la saturation des fonctions d'activation (comme la sigmoïde ou la tangente hyperbolique) et permette au modèle d'apprendre efficacement. Voici quelques méthodes recommandées :

- **Initialisation de Xavier/Glorot** : Elle est généralement utilisée pour les réseaux avec des fonctions d'activation symétriques comme la sigmoïde et la tangente hyperbolique. Elle aide à éviter la saturation en équilibrant la variance des sorties de chaque couche.
- **Initialisation de He** : Adaptée aux réseaux utilisant des fonctions d'activation ReLU. Elle a été conçue pour éviter la saturation des neurones tout en maintenant une bonne propagation des gradients.

Les méthodes comme l'initialisation aléatoire uniforme, normale, ou de zéro ne sont pas aussi efficaces pour éviter la saturation et peuvent ralentir l'apprentissage ou mener à des problèmes de convergence.

La réponse correcte est : Initialisation par Xavier.

Question 10

Non répondue

Noté sur 1,00

Que se passe-t-il si l'erreur augmente lors de l'adaptation du pas d'apprentissage dans une solution adaptative ?

Veuillez choisir une réponse.

- ☐ a. Le pas est réduit.
- ☐ b. Le pas est augmenté.
- ☐ c. Le pas est constant.
- ☐ d. L'algorithme est réinitialisé.

Si l'erreur augmente lors de l'adaptation du pas d'apprentissage dans une solution adaptative, cela peut signifier que le taux d'apprentissage est trop élevé. Cela peut entraîner des oscillations dans l'optimisation ou même une divergence, où l'algorithme "sauterait" autour de la solution optimale sans pouvoir la trouver. Il est aussi possible que l'algorithme d'adaptation du taux d'apprentissage soit mal paramétré.

Voici quelques pistes pour résoudre ce problème :

- Réduire le pas d'apprentissage pour éviter les oscillations.
- Utiliser des méthodes de régularisation pour rendre l'apprentissage plus stable.
- Vérifier l'initialisation des poids et des paramètres d'adaptation.
- Essayer un autre algorithme d'optimisation si nécessaire.

La réponse correcte est : Le pas est réduit.

◀ TP4: Réseaux à convolution pour la classification d'images

Aller à...

Visualisation ►